

03_matriz_final

May 9, 2026

1 Matriz Final de Clustering — Preprocessing y Escalado

Objetivo: Transformar `features_capa1.csv` y `features_capa2.csv` en matrices numéricas listas para K-Means.

1.1 Transformaciones aplicadas

Tipo de feature	Transformación	Razón
Binarias (0/1) <code>antiguedad_anos</code>	Ninguna StandardScaler	Ya están en escala comparable Variable continua con escala diferente
<code>sector_ciiu_macro</code>	One-hot encoding	Catagórica nominal — sin orden inherente
<code>region</code>	One-hot encoding	Catagórica nominal — sin orden inherente
<code>log_empleados</code> , <code>log_ingresos</code> , <code>log_activos</code>	StandardScaler	Continua, ya aplicado log previamente
<code>segmento</code>	StandardScaler	Ordinal 1-4 — tratado como continua
<code>liquidez_corriente</code> , <code>margen_operacional</code>	Imputar mediana → StandardScaler	Financieras con 5 nulos

1.2 Outputs

- `matriz_capa1.csv` — N=175 filas, sin `es_cliente_fpa`
- `matriz_capa2.csv` — N=92 filas, sin `es_cliente_fpa`
- `labels_capa1.csv` — identificadores + `es_cliente_fpa` para validación externa
- `labels_capa2.csv` — ídem para Capa 2

```
[1]: # Limitar hilos de OpenBLAS/OMP para evitar fallo de asignación de memoria RAM
import os
os.environ['OPENBLAS_NUM_THREADS'] = '1'
os.environ['OMP_NUM_THREADS'] = '1'
os.environ['MKL_NUM_THREADS'] = '1'

# Imports
import pandas as pd
```

```

import numpy as np
from sklearn.preprocessing import StandardScaler
from pathlib import Path
import warnings

warnings.filterwarnings('ignore')

ROOT      = Path('e:/TESIS MAESTRIA/Desarrollo_clustering_maestria')
PATH_C1   = ROOT / '03_feature_engineering/outputs/features_capa1.csv'
PATH_C2   = ROOT / '03_feature_engineering/outputs/features_capa2.csv'
PATH_OUTPUT = ROOT / '03_feature_engineering/outputs'

print('Imports y rutas OK.')

```

Imports y rutas OK.

1.3 1. Cargar features

```

[2]: capa1 = pd.read_csv(PATH_C1, dtype={'RUC': str})
     capa2 = pd.read_csv(PATH_C2, dtype={'RUC': str})

print(f'Capa 1: {capa1.shape} - clientes FPA: {capa1["es_cliente_fpa"].sum()}')
print(f'Capa 2: {capa2.shape} - clientes FPA: {capa2["es_cliente_fpa"].sum()}')

```

Capa 1: (167, 13) - clientes FPA: 53

Capa 2: (87, 20) - clientes FPA: 37

1.4 2. Función de preprocessing

Se aplica el mismo pipeline a Capa 1 y Capa 2.

```

[3]: def build_matriz(df: pd.DataFrame, nombre_capa: str) -> pd.DataFrame:
     """Transforma features_capaN en matriz numérica escalada para clustering."""
     df = df.copy()

     # Paso A: Separar labels e identificadores (NO van al modelo)
     COLS_META = ['name_norm', 'RUC', 'source_label', 'source_winner',
                  'es_cliente_fpa', 'anio']
     cols_meta_presentes = [c for c in COLS_META if c in df.columns]
     labels = df[cols_meta_presentes].copy()
     df_feat = df.drop(columns=cols_meta_presentes)

     # Paso B: Imputar nulos con mediana (solo variables financieras)
     COLS_IMPUTE = ['segmento', 'liquidez_corriente', 'margen_operacional']
     for col in COLS_IMPUTE:
         if col in df_feat.columns and df_feat[col].isna().sum() > 0:
             mediana = df_feat[col].median()
             n_nulos = df_feat[col].isna().sum()

```

```

        df_feat[col] = df_feat[col].fillna(media)
        print(f' [{nombre_capa}] Imputados {n_nulos} nulos en "{col}" con
↳media={media:.2f}')

# Paso C: One-hot encoding categóricas
COLS_OHE = ['sector_ciu_macro', 'region']
for col in COLS_OHE:
    if col in df_feat.columns:
        dummies = pd.get_dummies(df_feat[col], prefix=col,
↳drop_first=False, dtype=int)
        df_feat = pd.concat([df_feat.drop(columns=col), dummies], axis=1)

# Paso D: Verificar que no haya nulos antes de escalar
n_nulos_total = df_feat.isna().sum().sum()
if n_nulos_total > 0:
    print(f' ADVERTENCIA [{nombre_capa}]: {n_nulos_total} nulos residuales
↳antes de escalar:')
    print(df_feat.isna().sum()[df_feat.isna().sum() > 0])

# Paso E: StandardScaler para variables continuas
# Las binarias (0/1) y los one-hot NO se escalan.
COLS_BINARIAS = [
    'tipo_sociedad', 'obligado_contabilidad', 'es_agente_retencion',
    'es_contribuyente_especial', 'estado_activo'
]
# Identificar columnas one-hot
cols_ohe = [c for c in df_feat.columns
    if c.startswith('sector_ciu_macro') or c.
↳startswith('region_')]
# No escalar = binarias + one-hot
COLS_NO_ESCALAR = set(COLS_BINARIAS + cols_ohe)
# Escalar = resto (continuas)
COLS_ESCALAR = [c for c in df_feat.columns if c not in COLS_NO_ESCALAR]

scaler = StandardScaler()
df_feat[COLS_ESCALAR] = scaler.fit_transform(df_feat[COLS_ESCALAR])

print(f' [{nombre_capa}] Columnas escaladas: {COLS_ESCALAR}')
print(f' [{nombre_capa}] Columnas NO escaladas: {list(COLS_NO_ESCALAR)}')
print(f' [{nombre_capa}] Dimensión matriz: {df_feat.shape}')
print()

return df_feat, labels

print('Función build_matriz definida correctamente.')
```

Función build_matriz definida correctamente.

1.5 3. Construir matrices de Capa 1 y Capa 2

```
[4]: print('=== Procesando Capa 1 (N=175) ===')
matriz_c1, labels_c1 = build_matriz(capa1, 'Capa1')

print('=== Procesando Capa 2 (N=92) ===')
matriz_c2, labels_c2 = build_matriz(capa2, 'Capa2')

# Verificación final: no debe haber NaN ni infinitos
for nombre, mat in [('Capa 1', matriz_c1), ('Capa 2', matriz_c2)]:
    n_nan = np.isnan(mat.values.astype(float)).sum()
    n_inf = np.isinf(mat.values.astype(float)).sum()
    print(f'{nombre}: {mat.shape} - NaN={n_nan}, Inf={n_inf}')
```

```
=== Procesando Capa 1 (N=175) ===
[Capa1] Columnas escaladas: ['antiguedad_anos']
[Capa1] Columnas NO escaladas: ['tipo_sociedad', 'sector_ciiu_macro_C',
'region_Resto', 'es_agente_retencion', 'sector_ciiu_macro_M',
'sector_ciiu_macro_G', 'region_Pichincha', 'sector_ciiu_macro_K',
'es_contribuyente_especial', 'obligado_contabilidad', 'region_Guayas',
'estado_activo', 'sector_ciiu_macro_S', 'sector_ciiu_macro_OTRO']
[Capa1] Dimensión matriz: (167, 15)
```

```
=== Procesando Capa 2 (N=92) ===
[Capa2] Imputados 1 nulos en "segmento" con mediana=4.00
[Capa2] Imputados 5 nulos en "liquidez_corriente" con mediana=1.46
[Capa2] Imputados 5 nulos en "margen_operacional" con mediana=0.00
[Capa2] Columnas escaladas: ['antiguedad_anos', 'log_empleados',
'log_ingresos', 'log_activos', 'segmento', 'liquidez_corriente',
'margen_operacional']
[Capa2] Columnas NO escaladas: ['tipo_sociedad', 'sector_ciiu_macro_C',
'region_Resto', 'es_agente_retencion', 'sector_ciiu_macro_M',
'sector_ciiu_macro_G', 'region_Pichincha', 'sector_ciiu_macro_K',
'es_contribuyente_especial', 'obligado_contabilidad', 'region_Guayas',
'estado_activo', 'sector_ciiu_macro_S', 'sector_ciiu_macro_OTRO']
[Capa2] Dimensión matriz: (87, 21)
```

Capa 1: (167, 15) - NaN=0, Inf=0

Capa 2: (87, 21) - NaN=0, Inf=0

1.6 4. Exportar matrices y labels

```
[5]: # Matrices (solo features numéricas - sin es_cliente_fpa)
out_m1 = PATH_OUTPUT / 'matriz_capa1.csv'
out_m2 = PATH_OUTPUT / 'matriz_capa2.csv'
out_l1 = PATH_OUTPUT / 'labels_capa1.csv'
out_l2 = PATH_OUTPUT / 'labels_capa2.csv'
```

```

matriz_c1.to_csv(out_m1, index=False)
matriz_c2.to_csv(out_m2, index=False)
labels_c1.to_csv(out_l1, index=False)
labels_c2.to_csv(out_l2, index=False)

print(f' matriz_capa1.csv - {matriz_c1.shape}')
print(f' matriz_capa2.csv - {matriz_c2.shape}')
print(f' labels_capa1.csv - {labels_c1.shape}')
print(f' labels_capa2.csv - {labels_c2.shape}')
print()
print('Columnas de matriz_capa1:', list(matriz_c1.columns))
print()
print('Columnas de matriz_capa2:', list(matriz_c2.columns))

```

```

matriz_capa1.csv - (167, 15)
matriz_capa2.csv - (87, 21)
labels_capa1.csv - (167, 5)
labels_capa2.csv - (87, 6)

```

```

Columnas de matriz_capa1: ['tipo_sociedad', 'obligado_contabilidad',
'es_agente_retencion', 'es_contribuyente_especial', 'estado_activo',
'antiguedad_anos', 'sector_ciiu_macro_C', 'sector_ciiu_macro_G',
'sector_ciiu_macro_K', 'sector_ciiu_macro_M', 'sector_ciiu_macro_OTRO',
'sector_ciiu_macro_S', 'region_Guayas', 'region_Pichincha', 'region_Resto']

```

```

Columnas de matriz_capa2: ['tipo_sociedad', 'obligado_contabilidad',
'es_agente_retencion', 'es_contribuyente_especial', 'estado_activo',
'antiguedad_anos', 'log_empleados', 'log_ingresos', 'log_activos', 'segmento',
'liquidez_corriente', 'margen_operacional', 'sector_ciiu_macro_C',
'sector_ciiu_macro_G', 'sector_ciiu_macro_K', 'sector_ciiu_macro_M',
'sector_ciiu_macro_OTRO', 'sector_ciiu_macro_S', 'region_Guayas',
'region_Pichincha', 'region_Resto']

```